

Mendoza Sánchez Javier Alberto. 3TM3

Aplicaciones distribuidas

Evidencia Practica 3

1. mascaracteres

POST http://localhost:3000/mascaracteres

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2   "cad1": "Aplicaciones",
3   "cad2": "hola"
4 }
```

Body Cookies Headers (7) Test Results 200 OK 5 ms 263 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "resultado": "Aplicaciones"
3 }
```

POST http://localhost:3000/mascaracteres

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2   "cad1": "distribuidas",
3   "cad2": "Aplicaciones"
4 }
```

Body Cookies Headers (7) Test Results 200 OK 7 ms 263 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "resultado": "distribuidas"
3 }
```

2. menoscaracteres

REST client interface showing a POST request to `http://localhost:3000/menoscaracteres`. The request body is a JSON object:

```
1 {
2   "cad1": "hola",
3   "cad2": "Aplicaciones"
4 }
```

The response is a JSON object:

```
1 {
2   "resultado": "hola"
3 }
```

Status: 200 OK, Time: 8 ms, Size: 255 B.

3. numcaracteres

REST client interface showing a POST request to `http://localhost:3000/numcaracteres`. The request body is a JSON object:

```
1 {
2   "cadena": "ferrocarril"
3 }
```

The response is a JSON object:

```
1 {
2   "cadena": "ferrocarril",
3   "numCaracteres": 11
4 }
```

Status: 200 OK, Time: 6 ms, Size: 278 B.

4. palindroma

POST http://localhost:3000/palindroma

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1  
2 "cadena": "ferrocarril"  
3  
4  
5  
6  
7
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 8 ms Size: 278 B Save Response

Pretty Raw Preview Visualize JSON

```
1  
2 "cadena": "ferrocarril",  
3 "palindroma": false  
4
```

POST http://localhost:3000/palindroma

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

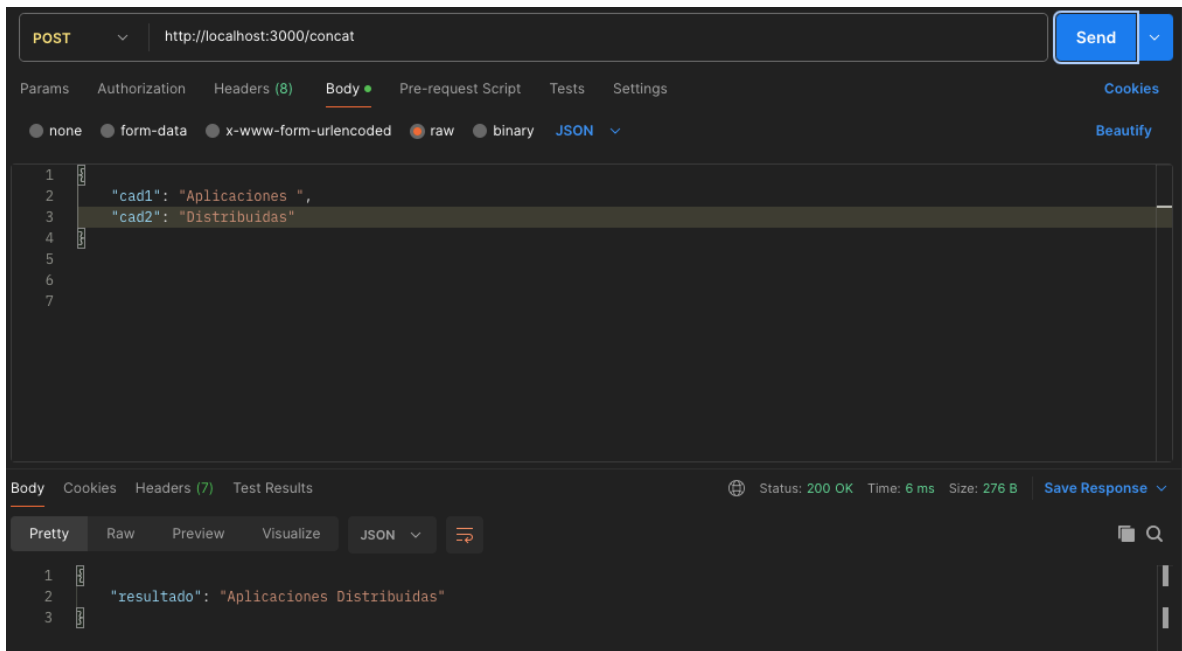
```
1  
2 "cadena": "ana"  
3  
4  
5  
6  
7
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 5 ms Size: 269 B Save Response

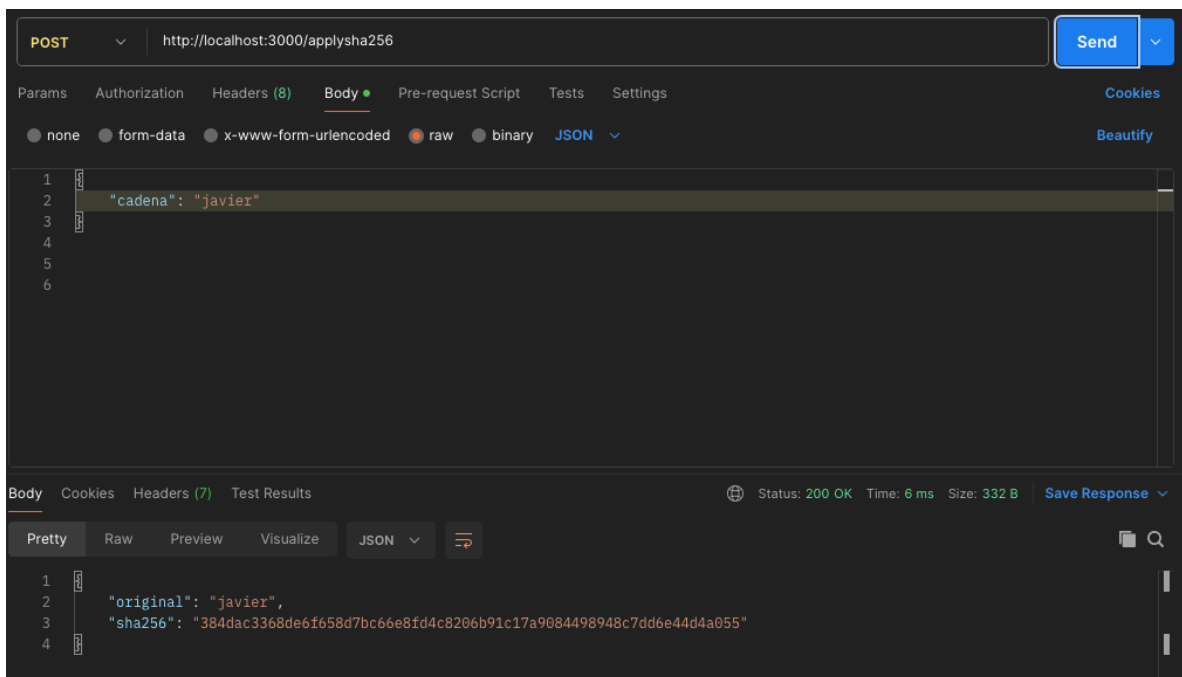
Pretty Raw Preview Visualize JSON

```
1  
2 "cadena": "ana",  
3 "palindroma": true  
4
```

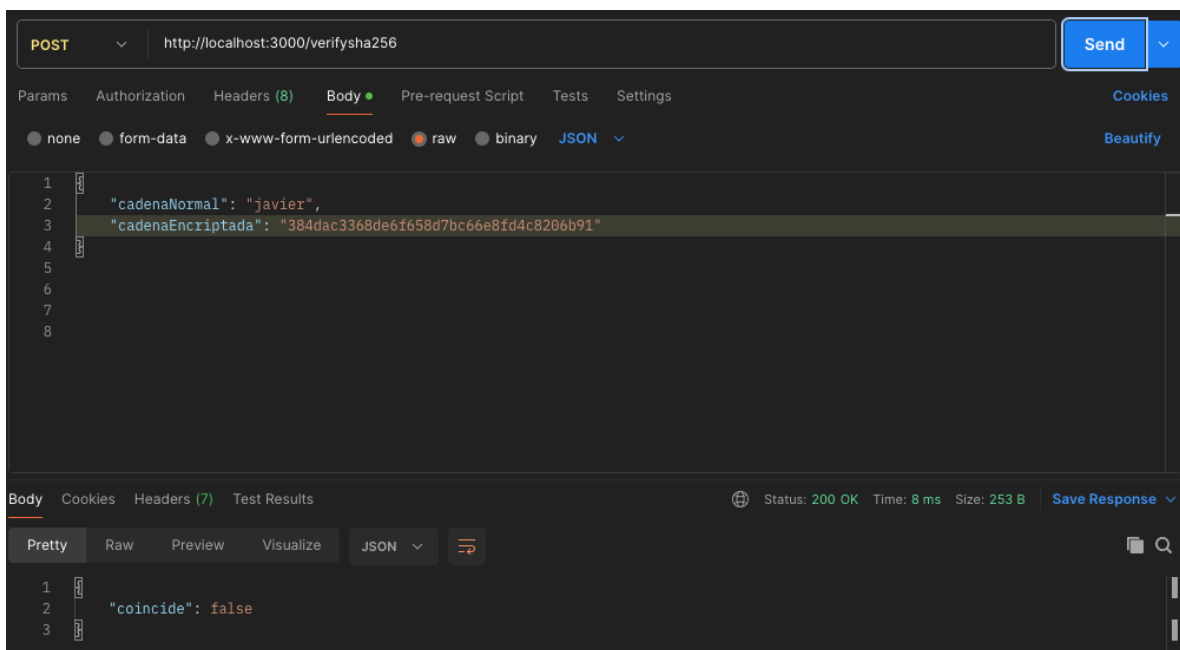
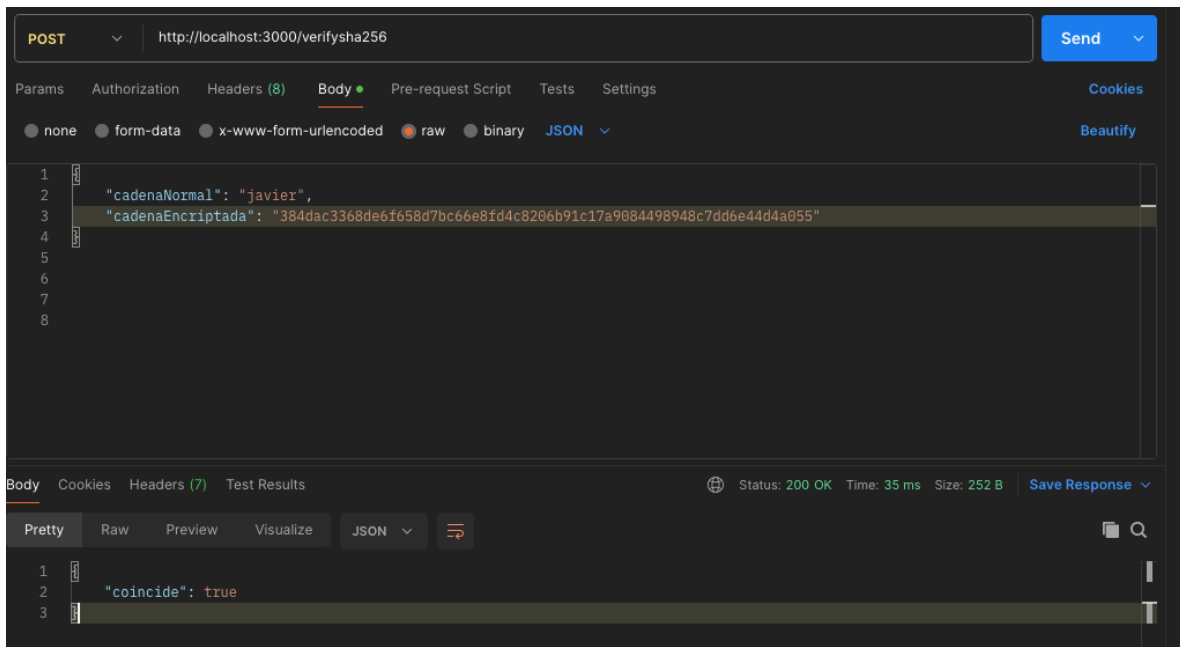
5. concat



6. applysha256



7. verifysha256



Código

```
const express = require("express");
```

```
const crypto = require("crypto");
```

```
const app = express();
```

```
app.use(express.json());
```

```
/* -----
```

```
    Función auxiliar para validar cadenas
```

```
-----*/
```

```
function validarCadenas(req, res, campos) {
```

```
    for (let campo of campos) {
```

```
        if (!(campo in req.body)) {
```

```
            res.json({
```

```
                error: `El parámetro '${campo}' es requerido`
```

```
            });
```

```
            return false;
```

```
        }
```

```
        if (typeof req.body[campo] !== "string") {
```

```
            res.json({
```

```
                error: `El parámetro '${campo}' debe ser una cadena`
```

```
            });
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

```

/* -----
i. mascharacteres
-----*/

app.post("/mascharacteres", (req, res) => {

  if (!validarCadenas(req, res, ["cad1", "cad2"])) return;

  const { cad1, cad2 } = req.body;

  const resultado = cad1.length >= cad2.length ? cad1 : cad2;

  res.json({ resultado });

});

```

```

/* -----
ii. menoscharacteres
-----*/

app.post("/menoscharacteres", (req, res) => {

  if (!validarCadenas(req, res, ["cad1", "cad2"])) return;

  const { cad1, cad2 } = req.body;

  const resultado = cad1.length <= cad2.length ? cad1 : cad2;

```

```

    res.json({ resultado });
  });

  /* -----
  iii. numcaracteres
  -----*/

  app.post("/numcaracteres", (req, res) => {

    if (!validarCadenas(req, res, ["cadena"])) return;

    res.json({

      cadena: req.body.cadena,

      numCaracteres: req.body.cadena.length

    });

  });

  /* -----
  iv. palindroma
  -----*/

  app.post("/palindroma", (req, res) => {

    if (!validarCadenas(req, res, ["cadena"])) return;

    const cad = req.body.cadena.toLowerCase();

    const invertida = cad.split("").reverse().join("");

```

```

    res.json({
        cadena: req.body.cadena,
        palindroma: cad === invertida
    });
});

/* -----
v. concat
-----*/

app.post("/concat", (req, res) => {
    if (!validarCadenas(req, res, ["cad1", "cad2"])) return;

    res.json({
        resultado: req.body.cad1 + req.body.cad2
    });
});

/* -----
vi. applysha256
-----*/

app.post("/applysha256", (req, res) => {
    if (!validarCadenas(req, res, ["cadena"])) return;

```

```

const hash = crypto.createHash("sha256")

    .update(req.body.cadena)

    .digest("hex");

res.json({

    original: req.body.cadena,

    sha256: hash

});

});

/* -----

vii. verifysha256

-----*/

app.post("/verifysha256", (req, res) => {

    if (!validarCadenas(req, res, ["cadenaNormal", "cadenaEncriptada"])) return;

    const hash = crypto.createHash("sha256")

        .update(req.body.cadenaNormal)

        .digest("hex");

    res.json({

        coincide: hash === req.body.cadenaEncriptada

```

```
});
```

```
});
```

```
/* -----
```

```
  Servidor
```

```
-----*/
```

```
app.listen(3000, () => {
```

```
  console.log("Servidor ejecutándose en http://localhost:3000");
```

```
});
```